LangGraph Core Concepts

LLM Workflows, Graphs, State, Reducers & Execution Model

1 What is LangGraph?

Definition: LangGraph is an **orchestration framework** for building intelligent, stateful, and multi-step LLM workflows.

1.1 How It Works

- Given an LLM workflow to execute, LangGraph first represents that workflow as a **graph**.
- Every **node** in the graph represents a single task in the workflow (e.g., calling an LLM, calling a tool, making a decision).
- Nodes are connected to each other through **edges**, which define what task should execute next after a given task finishes.
- Once the graph is built, you provide input to the first node and trigger the graph. All nodes then execute automatically in the correct order until the workflow completes.

1.2 Additional Features Beyond Graph Construction

- **Parallel execution** – multiple nodes can run at the same time after a given node.
- **Loops** – execution can cycle back to a previous node, repeatedly if needed.
- **Branching** – based on a condition, execution can move to one of several possible next nodes.
- **Memory** – all tasks and conversation happening in the workflow can be recorded.
- **Resumability** – if a workflow breaks down at some task, execution can be resumed from that same point instead of starting over.

Conclusion: These core features make LangGraph an ideal candidate for building agentic and production-grade AI applications.

2 LLM Workflows

2.1 What is a Workflow?

A **workflow** is a series of tasks that are executed, in order, to achieve a goal.

Example: In an automated hiring process – draft the JD, post it, shortlist candidates, conduct interviews, and onboard the selected candidate. Executing this series of steps in the right order completes the hiring workflow.

2.2 What is an LLM Workflow?

An **LLM workflow** is a workflow where many of the tasks in the series depend on LLMs. For example, the automated hiring workflow is an LLM workflow because tasks like drafting the JD, shortlisting, and conducting interviews may all require an LLM.

- LLM workflows are step-by-step processes used to build complex LLM applications.
- Each step performs a distinct task such as prompting, reasoning, tool calling, memory access, or decision making.
- Workflows can be **linear, parallel, branched, or looped**, allowing for complex behaviors like multi-agent communication and tool-augmented reasoning.

Every application has its own unique workflow, but certain workflow **patterns** recur across many applications. Five common LLM workflow patterns are described below.

2.3 1. Prompt Chaining

Concept: The LLM is called multiple times **in sequence**, where the output of one call becomes the input to the next.

Example: Generating a detailed report on a topic –

1. The topic is sent to a first LLM call, which produces an **outline**.
2. The outline is sent to a second LLM call, which produces the **detailed report**.

Why use it: Useful when a complex task can be broken into smaller sub-tasks, executed one after another. A validation check can also be inserted between calls (e.g., "report should not exceed 5000 words"; if it does, exit the flow).

2.4 2. Routing

Concept: An LLM call acts as a **router** or decision-maker: it examines a task/query and decides which downstream LLM (or path) should handle it.

Example: A customer support chatbot receives a query. An LLM first classifies whether the query is technical, refund-related, or sales-related, then routes the request to the LLM best suited to handle that category.

2.5 3. Parallelization

Concept: A given task is broken down into multiple sub-tasks, all of which are executed **simultaneously**. Their results are then merged to produce a final outcome.

Example: A content moderation workflow for a video platform checks a single video from three independent angles at once:

- Does it follow community guidelines?
- Does it contain misinformation?
- Does it contain sexual content?

Since none of these checks depends on the others, all three LLM calls run in parallel, and an **aggregator** combines their results to decide whether the video should be published.

2.6 4. Orchestrator–Workers

Concept: Very similar to parallelization – a task is divided into multiple parallel sub-tasks. The key difference: in orchestrator–workers, the **nature of the sub-tasks is not known in advance**; it is decided **dynamically**.

Example: A research assistant that generates a detailed report on a given query. Where to search and what to search for depends entirely on the query:

- A scientific/technical query might be routed to Google Scholar for research papers.
- A social or political query might be routed to Google News.

An **orchestrator** LLM analyzes the incoming query and dynamically decides what task each **worker** LLM should perform. Results are still aggregated at the end, but task assignment varies with input – unlike parallelization, where sub-task roles are fixed in advance.

2.7 5. Evaluator–Optimizer

Concept: Used for tasks that cannot be perfectly completed in a single attempt (e.g., drafting an email, writing a blog, a poem, or a story) – tasks that benefit from **iteration**, much like a writer producing multiple drafts.

How it works:

1. A **generator LLM** produces a solution (e.g., a blog draft).
2. An **evaluator LLM** checks the solution against a defined evaluation criteria.
3. If the evaluator accepts it, the loop ends and the output is returned.
4. If rejected, the evaluator provides feedback; the generator produces a new solution based on that feedback.
5. This loop continues until the evaluator is satisfied.

3 Graphs, Nodes and Edges

This trio is arguably the most important core concept in LangGraph – it is the mechanism by which any LLM workflow gets represented as a graph.

3.1 Worked Example: An Essay-Writing Practice Platform

Consider a website that helps UPSC exam aspirants practice writing essays (a major, high-weightage component of the UPSC Mains exam):

1. The website generates a topic for the user.
2. The user writes and submits an essay on that topic.
3. The essay is evaluated from multiple perspectives and given a score.

4. If the score is above the cutoff, the user is congratulated and the flow ends.

5. If below the cutoff, feedback is provided and the user is given the option to rewrite the essay.

6. If the user chooses to rewrite, the flow loops back to the writing step; otherwise, it ends.

This high-level goal must first be converted into a sequence of **actionable steps** (generate topic → collect essay → evaluate → aggregate scores → give feedback → optionally loop back), which can then be represented directly as a graph in LangGraph.

In the essay evaluation step, three criteria are scored (each normalized out of 5, totaling 15, with an example threshold of 10 to pass):

- **Clarity of thought**
- **Depth of analysis** (including fact-checking)
- **Language** (vocabulary, grammar, tonality)

3.2 Key Observations

- Any workflow of this kind can be represented very naturally as a graph.
- A graph is made up of two things: **nodes** and **edges**.

3.3 Nodes

- Every node represents a **single task** in the workflow.
- Behind the scenes, every node is simply a **Python function**. Nothing more.
- A LangGraph graph is essentially a set of Python functions interconnected via edges.

3.4 Edges

- Edges tell us **when** to execute a node – i.e., which node should run next after a given node finishes.
- Nodes tell us *what* to do; edges tell us *when* to do it.
- Types of edges:
  - **Sequential** – one node follows another.
  - **Parallel** – multiple nodes execute together.
  - **Conditional** – branching based on a condition (flow goes one way or another).
  - **Looping** – flow cycles back to an earlier node.

Takeaway: The graph structure gives LangGraph the freedom to express sequential flow, parallel flow, branching, and looping – all through the combination of nodes (tasks) and edges (execution flow).

4 State

4.1 What is State?

Any LLM workflow needs certain pieces of data to guide and support its execution – data that:

1. Is **required** for the workflow to execute logically, and

2. **Evolves over time** as execution progresses.

This data is called **State**.

Definition: In LangGraph, state is a shared memory that flows through your workflow. It holds all the data being passed between nodes as your graph runs.

4.2 Example (UPSC Essay Workflow)

Data points that would belong in the state:

- Essay text
- Essay topic
- Score for depth
- Score for language
- Score for clarity
- Overall score

4.3 Key Properties of State

- **Shared:** At any moment, every node in the graph has access to the complete state.
- **Mutable:** A node receives the state as input, performs its execution, makes changes to it, and passes the updated state forward to the next node.
- **Evolving:** As execution proceeds through the graph, the state keeps changing.

4.4 Implementation

- State is implemented as a **TypedDict** – a special Python class/dictionary.
- You create an object of this class and add all required fields to it.
- A Pydantic object can also be used, though TypedDict is the more common choice.

5 Reducers

Reducers are closely tied to the concept of state.

5.1 The Problem

Recall that state is (1) accessible to all nodes, and (2) mutable – meaning any node can change it. By default, an update to a state value **replaces** the previous value.

5.2 Example 1: Simple Arithmetic Workflow (Update Works Fine)

1. Input: two numbers → stored in state as `first_number` and `second_number`.
2. Node 2: sums them → writes the sum into `result`.
3. Node 3: multiplies `result` by 2 → overwrites `result` again.

Here, each node correctly **replaces** (updates) the previous value of `result`. This works fine for this scenario.

5.3 Example 2: Chatbot Workflow (Update Fails)

Consider a simple chatbot workflow where a human and an LLM converse in a loop, with state key `messages`.

1. Human: "Hi, my name is Nitish" → stored in `messages`.
2. LLM replies: "Hi, how can I help you?" → **replaces** the previous message in `messages`.
3. Human asks: "Can you tell me my name?" → again **replaces** the prior message.

Now the LLM can never answer correctly, because the message where the user stated their name was **erased** from state when it got replaced. Simply updating (replacing) the state fails in this scenario.

5.4 The Solution: Reducers

Definition: A reducer in LangGraph defines **how** updates from nodes are applied to the shared state. Each key in the state can have its own reducer, which determines whether new data **replaces**, **merges with**, or **adds to (appends to)** the existing value.

For the chatbot example, instead of replacing, an **add**-type reducer should be used so that every new message is appended to the list of messages rather than overwriting the previous one.

5.5 Example 3: UPSC Essay Revisions

If a student writes multiple drafts of an essay across attempts, simply updating `essay_text` would cause earlier drafts to be lost. If the student wants to review their own progress across drafts (draft 1, draft 2, draft 3), the state update policy should **append** each new draft rather than replace the old one – again handled via an **add** reducer function.

Summary: Reducers are especially useful when building **parallel workflows**, where multiple nodes may write to the same state key and their updates need to be combined (replaced, merged, or appended) in a well-defined way.

6 LangGraph Execution Model

LangGraph's execution model is inspired by **Google Pregel**, a system designed for large-scale graph processing, used internally across many Google products.

6.1 Phase 1: Graph Definition

- Define the **nodes** and **edges**.
- Define the **state** (a TypedDict).

6.2 Phase 2: Compilation

- The graph is **compiled** (via a `compile()` call) to verify that its structure is logically correct.
- Checks for issues such as **orphan nodes** – nodes that aren't connected to any other node.

6.3 Phase 3: Execution

1. **Invocation:** The initial state is passed to the first node of the graph, which **activates** it.
2. **Activation:** The Python function attached to that node is called; it performs its work and applies a **partial update** to the state.
3. **Message Passing:** The updated state automatically flows, via the outgoing edge(s), to the next node(s), which then activate in the same way.

6.4 Supersteps

- Each round of node activation → execution → state update → message passing is called a **superstep** (not just a "step").
- **Why "super"?** Because a single round may actually consist of **multiple nodes executing in parallel** (e.g., three parallel nodes triggered together). Since more than one step can happen within a single round, it is called a superstep rather than a simple step.
- When multiple parallel nodes update state within a superstep, their updates are combined via the appropriate **reducers**.

6.5 Termination

The workflow stops when both of the following are true:

- There is no active node remaining, and
- No edge is currently passing a message.

Key takeaway: You never manually call one node, wait, then call the next. Once invoked, LangGraph internally handles activation, execution, state updates, and message passing across all nodes and supersteps automatically.

7 Summary of Core Concepts

Concept	Definition
LangGraph	An orchestration framework for building intelligent, stateful, multi-step LLM workflows by representing them as graphs

Concept	Definition
LLM Workflow	A step-by-step process, involving one or more LLM calls, executed to achieve a goal
Prompt Chaining	Sequential LLM calls, where output of one feeds into the next
Routing	An LLM decides which downstream path/LLM should handle a task
Parallelization	A task split into fixed, known sub-tasks executed simultaneously, then merged
Orchestrator–Workers	Like parallelization, but sub-tasks are decided dynamically based on input
Evaluator–Optimizer	A generator–evaluator loop that iterates until output meets a defined criteria
Node	A single task in the workflow; behind the scenes, a Python function
Edge	Defines execution order – when a node runs relative to others (sequential, parallel, conditional, or looped)
State	Shared, mutable memory that flows through the graph, holding all data needed for execution
Reducer	Defines how a node's update is applied to a state key: replace, merge, or add
Graph Definition	The step of defining nodes, edges, and state
Compilation	Validates the graph's structural correctness (e.g., no orphan nodes)
Superstep	One round of execution in the graph, which may involve one or more nodes running (possibly in parallel)
Message Passing	The mechanism by which updated state is sent along edges to the next node(s)